

Centralized Real-Time Queue Management Platform

Development Plan — `development.md`

1. Document Information

| Field | Details |

|---|---|

| Project | Centralized Real-Time Queue Management Platform |

| Document | development.md |

| Version | 1.0 |

| Status | Development Plan |

| Frontend | Next.js |

| Backend | Node.js |

| Database | PostgreSQL |

| Architecture | Multi-Tenant |

| Notifications | WhatsApp + SMS |

| UI | Shadcn UI |

| Primary E2E | Playwright |

| Additional E2E | Selenium |

2. Development Objective

The purpose of this document is to define how the approved product will be developed from an empty repository into a production-ready centralized queue management platform.

Development must follow a controlled, phase-based approach.

The project should not be built as one large feature.

Each phase must produce a stable, testable, usable result.

``text

Requirements



Project Setup



Phase 1



Test + Secure



Publishable Checkpoint



Phase 2



Test + Secure



Publishable Checkpoint



Phase 3



Test + Secure



Publishable Checkpoint



Phase 4



Test + Secure



Publishable Checkpoint



Production

3. Development Principles

The project must follow these principles:

3.1 Understand Before Coding

The development agent must understand:

prd.md

techstack.md

database.md

architecture.md

development.md

before implementing major features.

3.2 Build in Phases

Features must be implemented in controlled phases.

Each phase must have:

- Clear scope.
 - Clear acceptance criteria.
 - Tests.
 - Security checks.
 - Responsive checks.
 - Definition of DONE.
 - Stable Git checkpoint.
-

3.3 Backend Is the Source of Truth

Queue-related business logic must remain on the backend.

The frontend must not independently determine authoritative:

- Queue positions.
- Queue ordering.
- Queue completion.
- Queue permissions.
- Tenant access.

- Waiting-time calculations.
-

3.4 Security From Day One

Security must not be added at the end.

Every phase must consider:

- Authentication.
 - Authorization.
 - Tenant isolation.
 - Input validation.
 - Rate limiting.
 - Secret management.
 - Secure API design.
 - Secure error handling.
-

3.5 Test While Building

Testing must happen during development rather than after the entire application is complete.

Build Feature



Test Feature



Fix Issues



Security Check



Responsive Check



DONE

4. Project Repository Structure

The project should use the approved monorepo approach if confirmed during scaffold setup.

Proposed structure:

project/

|

└─ apps/

| └─ web/

| └─ api/

|

| └─ packages/

| └─ ui/

| └─ types/

| └─ validation/

| └─ config/

|

| └─ tests/

|

| └─ .gitignore

| └─ .env.example

| └─ package.json

|

|— prd.md

|— techstack.md

|— database.md

|— architecture.md

└— development.md

The exact repository structure may be adjusted during scaffold creation.

5. Initial Project Setup

Before Phase 1 development begins, the following must be configured:

- Git repository.
- Branching strategy.
- Next.js application.
- Node.js backend.
- PostgreSQL connection.
- TypeScript.
- Linting.
- Formatting.
- Husky.
- Environment configuration.
- `.gitignore`.
- `.env.example`.
- Testing framework.

- Basic CI setup where applicable.
-

6. Environment Setup

Environment configuration must be separated from source code.

At minimum, development should support:

`.env.local`

`.env.development`

`.env.test`

`.env.production`

Only the required environment files should be used according to the deployment setup.

Secrets must never be committed.

7. `.gitignore` Security

The repository must contain a proper `.gitignore`.

It must prevent secrets and generated files from being committed.

Examples:

`.env`

`.env.*`

`!.env.example`

`node_modules/`

`.next/`

`dist/`

`coverage/`

The final `.gitignore` should be reviewed before the first production deployment.

8. Environment Example File

An `.env.example` should document required configuration without containing real secrets.

Example:

`DATABASE_URL=`

AUTH_SECRET=

WHATSAPP_ENABLED=

WHATSAPP_PROVIDER=

SMS_ENABLED=

SMS_PROVIDER=

Actual credentials must remain outside Git.

9. Git Branching Strategy

Development must not happen directly on the main branch for feature work.

Example:

main

|

|— feature/project-setup

|— feature/queue-management

|— feature/admin

|— feature/realtime

|— feature/notifications

└— feature/testing

The exact branch naming convention can be adjusted by the development team.

10. Git Commit Strategy

A Git commit should be created after every meaningful stable feature/checkpoint.

Do not create unnecessary commits for every tiny edit.

Example:

feat: initialize project structure

feat: implement tenant management

feat: implement queue joining

feat: implement queue completion

feat: implement realtime updates

feat: implement notifications

Every completed phase must end with a stable Git checkpoint.

11. Development Phase Overview

The project will be developed in **5 phases**.

Phase 1

Foundation + Multi-Tenant Core



Phase 2

Queue Management



Phase 3

Real-Time + Waiting Time



Phase 4

Notifications + Production Features



Phase 5

Hardening + Production Release

Each phase must be independently usable and publishable.

PHASE 1 — FOUNDATION + MULTI-TENANT CORE

12. Phase 1 Objective

Build the foundation of the application and establish the secure multi-tenant structure.

At the end of Phase 1:

- Application runs.
- Database works.
- Tenant structure works.
- Authentication works.
- Authorization works.
- Admin can access their tenant.
- Basic queue creation works.
- Admin can generate a QR.
- Users can access the queue entry page.
- Responsive foundation exists.

This creates the first publishable version of the platform.

13. Phase 1 Features

Implement:

- Project scaffold.
- Next.js frontend.
- Node.js backend.
- PostgreSQL.
- Database migrations.
- Tenant creation.
- User authentication.
- Role system.

- Tenant isolation.
 - Organization Admin dashboard.
 - Basic queue creation.
 - Admin QR generation.
 - Public QR queue page.
 - Basic registration form.
 - Responsive UI foundation.
-

14. Phase 1 QR Flow

Only the Organization Admin can generate a QR.

Admin Login



Admin Dashboard



Create Queue



Generate QR



Display QR

End user:

Scan QR



Open Queue Page



Enter Details

15. Phase 1 Security

Implement:

- Authentication.
- Role-based authorization.
- Tenant isolation.
- Backend validation.
- Secure session/token handling.
- Rate limiting for relevant public endpoints.
- Secure environment configuration.
- `.gitignore`.
- No secrets in Git.

16. Phase 1 Testing

Unit Tests

Test:

- Tenant logic.
- Authentication logic.
- Authorization logic.
- Queue creation.
- QR authorization.

Happy Flow

Admin Login



Create Queue



Generate QR



User Scans QR



Queue Page Opens

Negative Flow

Test:

Unauthorized User



Attempts Admin QR Generation



Request Rejected

Also test:

Tenant A



Attempts Tenant B Data



Access Rejected

17. Phase 1 Responsive Testing

Verify:

- Mobile.
- Tablet where applicable.
- Laptop.
- Desktop.

Especially test:

- QR queue page.
 - Registration form.
 - Admin dashboard.
 - QR generation interface.
-

18. Phase 1 DONE Definition

Phase 1 is DONE only when:

- Project builds successfully.
- Next.js application works.
- Node.js backend works.
- PostgreSQL works.
- Migrations work.
- Tenant creation works.
- Authentication works.
- Authorization works.
- Tenant isolation is tested.
- Admin can create a queue.
- Only authorized Admin can generate QR.
- User can open QR queue page.

- Registration form works.
 - API validation works.
 - Rate limiting is implemented where required.
 - `.gitignore` is configured.
 - Secrets are not committed.
 - Husky works.
 - Unit tests pass.
 - Happy flow passes.
 - Negative flow passes.
 - Mobile layout works.
 - Desktop layout works.
 - No critical bugs remain.
 - Git checkpoint created.
 - Phase can be deployed.
-

PHASE 2 — QUEUE MANAGEMENT

19. Phase 2 Objective

Implement the complete core queue engine.

At the end of Phase 2, the system must allow real people to:

Scan QR



Enter Details



Join Queue



Receive Position



Wait



Operator Marks DONE



User Leaves Active Queue



Remaining Users Move Forward

This is the first fully functional queue-management release.

20. Phase 2 Features

Implement:

- Queue joining.
 - Automatic queue ordering.
 - First-Come-First-Served.
 - Queue entry creation.
 - Queue position calculation.
 - Operator dashboard.
 - DONE operation.
 - User cancellation.
 - Cancellation confirmation.
 - Queue state management.
 - Admin manual queue reordering.
 - Queue event recording.
-

21. Phase 2 Automatic Queue Numbering

Example:

Person 1 → #1

Person 2 → #2

Person 3 → #3

Person 4 → #4

The backend must assign the ordering.

Users must not be able to choose their own queue number.

22. Phase 2 DONE Flow

Example:

Before:

#1 A

#2 B

#3 C

#4 D

Operator clicks:

DONE

Result:

A → COMPLETED

#1 B

#2 C

#3 D

Historical information about A must remain available.

23. Phase 2 Admin Reordering

Organization Admin can modify queue ordering.

Example:

Before:

#1 A

#2 B

#3 C

#4 D

Admin moves D:

#1 A

#2 D

#3 B

#4 C

This is the mechanism used for VIP/priority handling.

24. Phase 2 User Cancellation

User selects:

Leave Queue

The application must first display:

You will lose your current queue position.

Are you sure?

After confirmation:

WAITING

↓

CANCELLED

Remaining active users move forward.

25. Phase 2 Testing

Happy Flow

User A joins → #1

User B joins → #2

User C joins → #3

Operator completes A

B becomes #1

C becomes #2

Negative Flow

Test:

- User submits invalid details.
 - User attempts duplicate join where duplicate rules apply.
 - Unauthorized operator action.
 - Unauthorized admin action.
 - User attempts to modify another user's queue entry.
 - Invalid queue ID.
 - Closed queue join attempt.
-

26. Phase 2 Concurrency Testing

Test simultaneous operations:

User A → JOIN

User B → JOIN

User C → JOIN

Ensure:

#1

#2

#3

and never:

#1

#1

#2

Also test:

Operator → DONE

Admin → REORDER

User → JOIN

occurring close together.

27. Phase 2 DONE Definition

Phase 2 is DONE only when:

- Users can join queues.
- Queue positions are assigned automatically.
- FCFS works.
- Operator can mark entries DONE.
- Completed users leave active queue.
- Remaining users move forward.
- Admin can reorder users.
- User cancellation works.
- Cancellation warning works.
- Queue events are stored.
- Concurrency is handled.
- Unit tests pass.
- API tests pass.
- Happy flow passes.
- Negative flow passes.
- Responsive testing passes.
- Security checks pass.
- Git checkpoint created.
- Phase can be deployed.

PHASE 3 — REAL-TIME + WAITING TIME

28. Phase 3 Objective

Make the queue genuinely real-time and introduce approximate waiting-time calculation.

At the end of Phase 3:

Operator clicks DONE



Backend updates queue



Real-Time Event



All connected users



Positions update automatically

No manual refresh should be required.

29. Phase 3 Real-Time Features

Implement:

- Real-time queue updates.
 - Real-time position changes.
 - Real-time queue clearing.
 - Real-time admin reordering.
 - Connection recovery.
 - Latest-state synchronization.
 - Real-time event handling.
-

30. Phase 3 Real-Time Example

Before:

User A → #1

User B → #2

User C → #3

Operator:

A → DONE

Without refreshing:

User B → #1

User C → #2

The frontend updates automatically.

31. Phase 3 Waiting-Time Calculation

Use actual previous service durations.

Example:

Completed services:

8 min

10 min

7 min

9 min

Average:

8.5 minutes

If the current user has 3 people ahead:

Estimated Wait

$\approx 3 \times 8.5$

≈ 25.5 minutes

The application should present this as an approximation.

32. Phase 3 Waiting-Time Updates

Recalculate when relevant events occur:

- New user joins.
- Person ahead completes.
- Person ahead cancels.
- Admin reorders the queue.
- New service-time data becomes available.

Example:

Position #4

Estimated Wait: ~26 min

Person ahead completes

Position #3

Estimated Wait: ~17 min

33. Phase 3 Insufficient Data

If there is insufficient historical service data:

Estimated Wait

Calculating...

or another approved fallback should be displayed.

The system must not present a misleading precise estimate.

34. Phase 3 Testing

Happy Flow

User A joins

User B joins

User C joins

Operator completes A

B automatically changes:

#2 → #1

C automatically changes:

#3 → #2

Negative Flow

Test:

- Real-time connection interruption.
- Reconnection.
- Out-of-date client state.
- Multiple simultaneous queue changes.
- Invalid real-time event.
- Unauthorized real-time subscription.

35. Phase 3 DONE Definition

Phase 3 is DONE only when:

- Real-time queue updates work.
 - No manual refresh is required.
 - DONE triggers real-time updates.
 - Admin reorder triggers real-time updates.
 - Cancellation triggers real-time updates.
 - Queue clearing works.
 - Connection recovery works.
 - Backend remains source of truth.
 - Waiting-time calculation works.
 - Historical service durations are recorded.
 - Insufficient-data behavior is handled.
 - Unit tests pass.
 - Real-time tests pass.
 - Happy flow passes.
 - Negative flow passes.
 - Mobile testing passes.
 - Desktop testing passes.
 - Security checks pass.
 - Git checkpoint created.
 - Phase can be deployed.
-

PHASE 4 — NOTIFICATIONS + PRODUCTION FEATURES

36. Phase 4 Objective

Implement WhatsApp and SMS notifications and complete the major production-facing features.

37. Phase 4 Notification Features

Implement:

- WhatsApp integration.
 - SMS integration.
 - Notification service.
 - Notification event processing.
 - Notification delivery tracking.
 - Environment-based configuration.
 - Notification failure handling.
 - Duplicate-notification prevention.
-

38. Phase 4 Notification Flow

Queue Event



Database Commit



Notification Service



WhatsApp



SMS

Both channels must be independently supported.

SMS must not automatically be treated only as a fallback.

39. Phase 4 Notification Configuration

Notification functionality must be configurable.

Example:

WHATSAPP_ENABLED=true

SMS_ENABLED=true

Provider credentials must be stored securely.

Actual provider selection must remain configurable.

40. Phase 4 Notification Events

Potential events:

QUEUE_JOINED

TURN_APPROACHING

TURN_AVAILABLE

QUEUE_CLEARED

QUEUE_CLOSED

The final notification rules must follow the approved product requirements.

Every real-time position change does not necessarily need an external WhatsApp/SMS message.

41. Phase 4 Queue Closure/Pause

Implement queue state handling where required:

ACTIVE

PAUSED

CLOSED

When a queue closes:

- New users cannot join.
 - Existing users receive appropriate state information.
 - Required notifications are generated.
-

42. Phase 4 Public Queue Status

Users should be able to view their current queue status through a secure public session/status mechanism where required.

Example:

Your Position

#3

People Ahead

2

Estimated Wait

~17 minutes

The public status mechanism must not expose another user's private information.

43. Phase 4 Testing

Happy Flow

User joins



Queue progresses



Turn becomes available



WhatsApp sent



SMS sent



Delivery recorded

Negative Flow

Test:

- WhatsApp failure.
 - SMS failure.
 - Invalid provider response.
 - Duplicate notification event.
 - Missing configuration.
 - Expired public queue session.
 - Unauthorized queue access.
-

44. Phase 4 DONE Definition

Phase 4 is DONE only when:

- WhatsApp works.
- SMS works.
- Environment configuration works.
- Provider failures are handled.
- Delivery status is tracked.
- Duplicate notifications are prevented.
- Important queue events trigger correct notifications.
- Queue pause/closure behavior works where required.
- Secure public queue status works.
- Unit tests pass.
- Integration tests pass.
- E2E tests pass.

- Happy flow passes.
 - Negative flow passes.
 - Mobile testing passes.
 - Desktop testing passes.
 - Security checks pass.
 - Git checkpoint created.
 - Phase can be deployed.
-

PHASE 5 — HARDENING + PRODUCTION RELEASE

45. Phase 5 Objective

Prepare the complete platform for production.

This phase focuses on:

- Security.
- Performance.
- Testing.
- Reliability.
- Accessibility.
- Responsive behavior.
- Monitoring.
- Deployment.
- Documentation.

46. Phase 5 Security Review

Perform a full security review covering:

- Authentication.
- Authorization.
- Tenant isolation.
- Public QR access.
- Public queue status.
- API validation.
- Rate limiting.
- Session/token security.
- Database permissions.
- Secret management.
- `.gitignore`.
- Error exposure.
- Logging.
- Audit events.

47. Multi-Tenant Security Testing

Explicitly test:

Tenant A



Attempts Tenant B Queue



BLOCKED

Test:

Tenant A Admin



Attempts Tenant B User Data



BLOCKED

Test:

Tenant A Public QR



Attempts Access to Tenant B



BLOCKED

Tenant isolation must be verified at the API and database-access levels.

48. API Security Testing

Test:

- Unauthorized API requests.
 - Invalid tokens.
 - Expired sessions.
 - Incorrect roles.
 - Cross-tenant requests.
 - Malformed requests.
 - Rate-limit violations.
 - Invalid IDs.
 - Enumeration attempts.
 - Duplicate requests.
-

49. Queue Integrity Testing

Test:

50 Users



Users Join



#1 ... #50

Then complete users randomly.

Verify that:

- No duplicate active positions exist.
 - No active user disappears unexpectedly.
 - Completed users are not returned to the active queue.
 - Queue ordering remains consistent.
 - Admin reordering remains consistent.
 - Simultaneous joins remain safe.
-

50. End-to-End Testing

Playwright must cover critical user journeys.

Minimum happy flows:

Admin Login



Create Queue



Generate QR



User Joins



Multiple Users Join



Operator Completes User



Positions Update



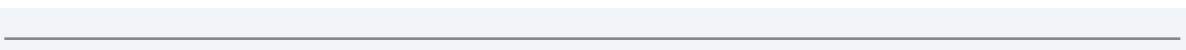
Estimated Wait Updates



Turn Becomes Available



Notification Triggered



51. Negative End-to-End Testing

Minimum negative scenarios:

Unauthorized Admin Action

Cross-Tenant Access

Invalid QR

Invalid Registration

Duplicate Join

Invalid Queue

Closed Queue Join

Unauthorized Queue Operation

Notification Failure

Real-Time Disconnect

52. Selenium Testing

Selenium should be used where required for additional browser compatibility/testing.

It should complement Playwright rather than unnecessarily duplicate the complete suite.

53. Responsive Testing

Final testing must include:

Mobile

Tablet

Laptop

Desktop

Verify:

- Layout.
- Forms.
- Queue status.
- Admin dashboard.
- Operator dashboard.
- Buttons.
- Dialogs.
- Tables/lists.
- Navigation.
- Error messages.
- Loading states.

No important functionality should break at supported viewport sizes.

54. Accessibility Review

Review:

- Keyboard navigation.
 - Focus states.
 - Form labels.
 - Error messages.
 - Contrast.
 - Touch target sizes.
 - Semantic structure.
 - Screen-reader compatibility where applicable.
-

55. Performance Review

Review:

- API response times.
- Database queries.
- Queue position queries.
- Queue updates.
- Real-time connections.
- Notification processing.
- Large queue behavior.
- Concurrent user joins.

- Concurrent operator actions.

The system must remain stable under realistic queue loads.

56. Database Review

Before production:

- Review indexes.
 - Review foreign keys.
 - Review constraints.
 - Review transactions.
 - Review tenant isolation.
 - Review migrations.
 - Review backup strategy.
 - Review recovery strategy.
 - Review data retention.
-

57. Production Configuration Review

Before release:

Production Environment



Secrets Configured



Database Configured



WhatsApp Configured



SMS Configured



Authentication Configured



Rate Limits Configured



Monitoring Configured

No development secrets should be used in production.

58. Production Deployment Checklist

Before production release:

- Production database created.
 - Database migrations tested.
 - Environment variables configured.
 - Secrets secured.
 - HTTPS enabled.
 - Authentication configured.
 - Authorization verified.
 - Tenant isolation tested.
 - Rate limiting enabled.
 - WhatsApp configured.
 - SMS configured.
 - Real-time system configured.
 - Logging enabled.
 - Monitoring enabled.
 - Backups configured.
 - Recovery process documented.
 - Production build succeeds.
 - E2E tests pass.
 - Security review passes.
 - Responsive review passes.
-

59. Production Release Criteria

The application must not be considered production-ready merely because the application starts successfully.

Production release requires:

Functionality

+

Security

+

Testing

+

Performance

+

Responsive UI

+

Data Integrity

+

Monitoring

+

60. Definition of DONE

The project-wide definition of DONE is:

A feature is DONE only when it is implemented, validated, tested, secured, responsive, documented where necessary, and integrated without breaking existing functionality.

A feature is NOT DONE if:

Code works locally

✗

but:

Tests fail

Security incomplete

Mobile broken

API unvalidated

Tenant isolation untested

61. Feature-Level DONE Checklist

Every meaningful feature must satisfy:

☐ Requirement understood

☐ Implementation complete

☐ Backend logic complete

☐ Frontend complete

☐ Validation complete

☐ Error handling complete

☐ Security reviewed

☐ Tenant isolation reviewed where applicable

☐ Unit tests complete

☐ Integration tests complete where required

☐ Happy flow tested

☐ Negative flow tested

☐ Mobile tested

☐ Desktop tested

☐ No lint/type errors

☐ No unused imports/variables

☐ Documentation updated where necessary

☐ Git checkpoint created

62. Code Quality Definition of DONE

Before considering a feature complete:

Lint



Type Check



Unit Tests



Integration Tests



E2E Tests



Build

All required checks must pass.

63. Husky Requirements

Husky should prevent common quality issues before commits.

Checks may include:

- Linting.
- Formatting.
- Type checking.
- Relevant unit tests.
- Other fast validation checks.

Heavy E2E suites should not necessarily run on every local commit if that creates an unreasonable development workflow.

Full E2E testing should run through the appropriate test/CI process.

64. Testing Pyramid

The project should follow:

E2E

/ \

Integration

/ \

Unit Tests

Unit tests should cover business logic.

Integration tests should cover important API/database interactions.

E2E tests should cover complete user journeys.

65. Regression Testing

When a feature is completed, existing functionality must be checked.

Example:

New Feature

↓

Feature Tests



Existing Test Suite



Build



Regression Check

A new feature must not silently break previously completed phases.

66. Documentation Maintenance

When implementation changes an architectural or database decision:

Code Change



Relevant .md Review



Update Documentation



Commit

The following documents must remain synchronized:

prd.md

techstack.md

database.md

architecture.md

development.md

67. Temporary Development Pages

Temporary UI preview pages may be created for design validation.

They must be:

- Clearly marked as temporary.
- Used for preview/testing only.
- Removed before production if not part of the final product.

68. No Unnecessary Complexity

The project should avoid unnecessary:

- Microservices.
- Databases.
- Infrastructure.
- Dependencies.
- Duplicate functionality.
- Duplicate testing frameworks.

The architecture should remain simple until real requirements justify additional complexity.

69. Change Management

If a requirement changes:

New Requirement



PRD Review



Impact Analysis



Database Review



Architecture Review



Development Plan Update



Implementation

Do not directly implement major requirement changes without reviewing their effect on the existing architecture.

70. Phase Completion Rule

A phase cannot be considered complete simply because the feature has been coded.

Each phase must satisfy:

Implementation



Testing

+

Security

+

Responsive UI

+

Documentation

+

Git Checkpoint

+

Deployability

Only then is the phase complete.

71. Final Development Sequence

The complete development sequence is:

STEP 1

Understand Documentation



STEP 2

Evaluate Final Technical Decisions



STEP 3

Create Project Scaffold



STEP 4

Configure Git + Husky + Environment



STEP 5

Phase 1

Foundation + Multi-Tenant Core



STEP 6

Test + Secure + Deploy



STEP 7

Phase 2

Queue Management



STEP 8

Test + Secure + Deploy



STEP 9

Phase 3

Real-Time + Waiting Time



STEP 10

Test + Secure + Deploy



STEP 11

Phase 4

Notifications + Production Features



STEP 12

Test + Secure + Deploy



STEP 13

Phase 5

Hardening + Production Release



STEP 14

72. Final Project Definition of DONE

The entire project is DONE only when:

- All PRD requirements are implemented.
- All approved features are working.
- Multi-tenancy works correctly.
- Tenant data is isolated.
- Admin can create queues.
- Only Admin can generate QR codes.
- Users can join through QR.
- Queue numbers are automatically assigned.
- FCFS works.
- Admin can manually change queue order.
- Users can leave the queue.
- Cancellation warning works.
- Operator can mark users DONE.
- Completed users leave the active queue.
- Remaining users move forward.
- Real-time updates work.
- Queue clearing notifications work.
- Estimated waiting time works.
- Historical service times are used for estimates.
- WhatsApp works.
- SMS works.
- Notification configuration works through environment/configuration.

- Notification delivery is tracked.
 - Authentication works.
 - Authorization works.
 - API protection works.
 - Rate limiting works.
 - Frontend validation works.
 - Backend validation works.
 - Database constraints work.
 - Unit tests pass.
 - Integration tests pass where required.
 - Playwright tests pass.
 - Selenium requirements are satisfied.
 - Happy flows pass.
 - Negative flows pass.
 - Mobile UI works.
 - Desktop UI works.
 - Husky checks work.
 - No secrets are committed.
 - `.gitignore` is secure.
 - Logging is implemented.
 - Monitoring is configured.
 - Database backup/recovery is addressed.
 - Production deployment succeeds.
 - Documentation is up to date.
 - Final Git release checkpoint is created.
-

73. Final Development Philosophy

The project should be developed with the following mindset:

Understand



Plan



Build



Validate



Test



Secure



Review



Deploy



Commit

Never:

Code



Hope it works



Deploy

Every meaningful feature must be complete across functionality, security, testing, responsiveness, and reliability before it is considered DONE.

END OF DEVELOPMENT DOCUMENT